Global Health Observatory

Project 3 Pitch — Full-Stack Application with Authentication

Group 6 · Moringa School · SDF-PT13M6 · July 2026

Team: Collins Kiptoo (Lead — Architecture & Auth) · Shawn Ochieng (Backend API) · Rhoda Kinoti (Auth Frontend) · Samuel Wanjau (Frontend Features & UI) · Wasaa Abdalla (Documentation & Presentation)

Part 1 — Business Problem Scenario

Global health data from the World Health Organization is publicly available, but it is buried in raw API responses that are difficult for non-technical people to explore. Our users are **students, researchers, and public-health workers** who track health indicators - life expectancy, immunization coverage, disease burden - across specific countries. Their goal is to monitor the same small set of indicator–country pairs over time: a researcher following measles immunization in Kenya and Uganda, or a student comparing life expectancy across East Africa for a thesis. In the current version of our dashboard, every visit starts from zero. There is no way to save a selection, so users repeat the same country and indicator look-ups on every session. If this need stays unmet, the tool remains a one-off lookup site rather than a workspace people return to - time is wasted on repeated navigation, ongoing monitoring has no continuity, and there is no reason for a user to come back.

The **Global Health Observatory** solves this by adding secure, personal accounts to our existing React + Flask dashboard. Any visitor can still browse the public data views, but a registered user can log in and save **favorites**; private indicator–country pairs that persist between sessions and load instantly on a personal “My Favorites” dashboard. Each user sees only their own favorites, enforced by the backend, while administrators retain the existing power to manage the curated catalogue of countries and indicators. The value is a personalised, trustworthy research companion: users stop re-doing the same searches, their watchlists are private to them, and the platform gains the account layer any real multi-user product needs.

Part 2 — Problem-Solving Process

Development stages

- **1. Define data models and relationships.**

  Unify authentication into a single *users* table with a *role* column ('user' | 'admin'), and make *Favorite* user-owned by adding a *user_id* foreign key alongside its country and indicator links, with a unique constraint on (user, country, indicator). Ship the schema change as a Flask-Migrate migration.

- **2. Implement token-based authentication.**

Build *POST /auth/register*, *POST /auth/login*, *POST /auth/logout* and *GET /auth/me*. Hash passwords with werkzeug.security (never stored in plaintext), validate email format and minimum password length, and issue stateless signed tokens.

- **3. Protect the Flask API.**

  Write *@login_required* and *@admin_required* decorators that verify the Bearer token and load the current user; apply ownership filtering so every favorites query is scoped to *user_id == current_user.id*, and restrict country/indicator writes to admins.

- **4. Build the React auth layer.**

  A shared Axios instance with a request interceptor that attaches the token and a response interceptor that wipes a rejected token on 401; *AuthContext* holding user state and hydrating from */auth/me* on page load; Login and Register pages with form handling and error states.

- **5. Add route protection and conditional rendering.**

  A *ProtectedRoute* wrapper redirects unauthenticated visitors from /favorites to /login (returning them after login), and the navbar renders a logged-in user chip or a login prompt based on auth state. Public data views stay open to everyone.

- **6. Test CRUD operations and the auth flow.**

  Postman pass over every endpoint, plus a two-account test to prove one user can never read, update, or delete another user's favorites — even by guessing IDs.

- **7. Style, finalize UI/UX, and document.**

  Align auth pages with the existing teal design system, polish loading/error states, update the README and ERD, and record the demo.

Authentication method and why

We chose **token-based authentication (JWT-style stateless signed tokens) over server-side sessions**.

The backend signs a compact token containing the user's id using itsdangerous' URLSafeTimedSerializer with the application secret key and an 8-hour expiry; the client sends it back on every request as an *Authorization: Bearer* header. Because our React frontend and Flask API are fully decoupled (separate servers in development, separate origins in production), stateless tokens fit better than session cookies: no server-side session store to manage, no cookie/CORS credential configuration between origins, and the same pattern extends cleanly to mobile or third-party clients. One auth system serves everyone — the role field on the user, not a second login flow, decides who gets admin power.

Conceptual plan — authentication flow

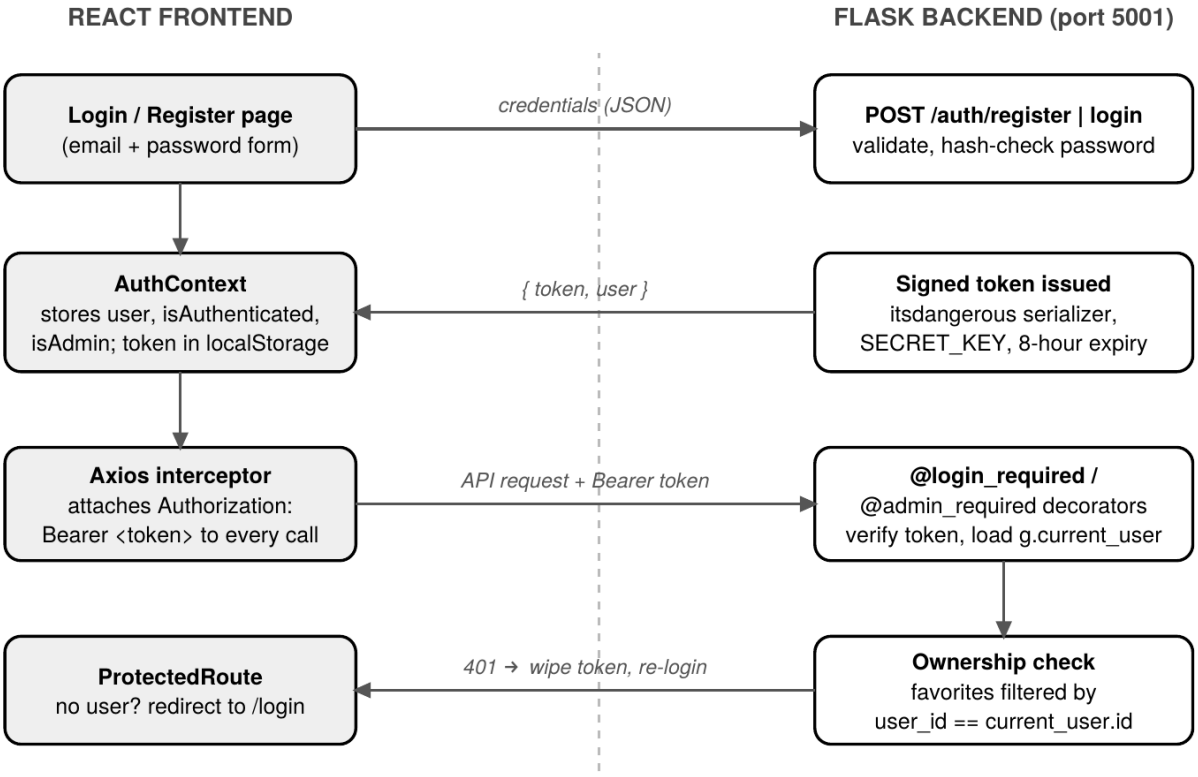
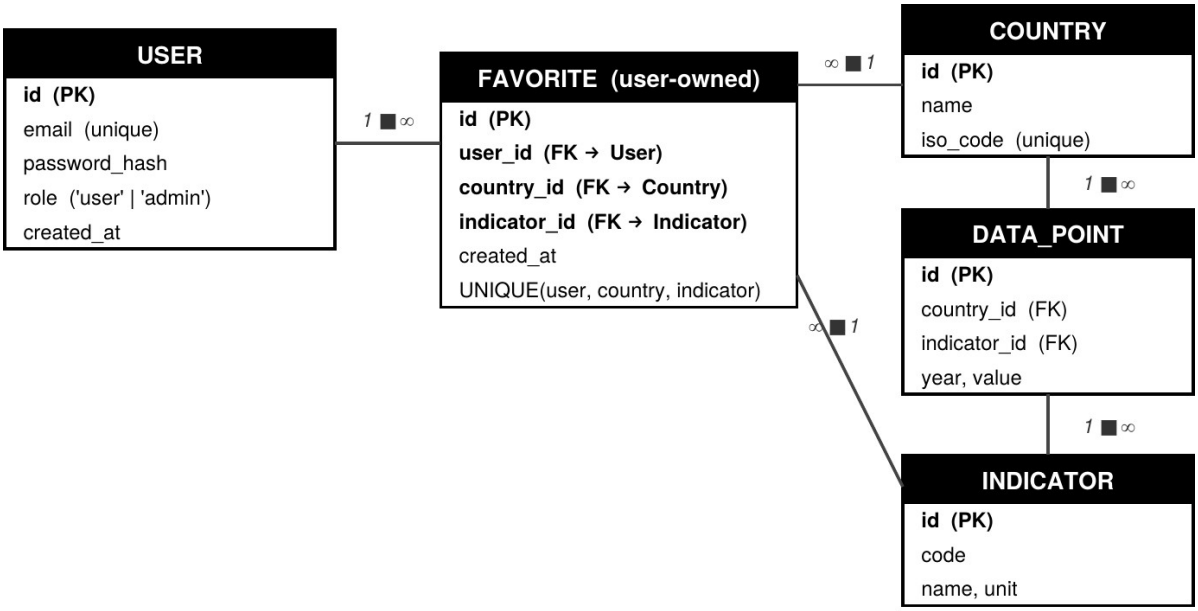


Figure 1 — How a request travels from the React client to a protected Flask route.

Conceptual plan — data model (ERD)



Access rule: every /favorites query is filtered by the logged-in user's id — enforced in Flask, not the UI.

Figure 2 — Entity relationships: User 1→∞ Favorite; each Favorite links one Country and one Indicator.

Tools and libraries

Layer	Choices
Frontend	React 19, Vite, React Router 6, Axios, Recharts, CSS design system (teal theme)
Backend	Flask, SQLAlchemy, Flask-Migrate, Flask-CORS, marshmallow
Auth & security	itsdangerous (signed tokens), werkzeug.security (password hashing), env-based SECRET_KEY
Database	PostgreSQL (SQLite fallback in local development)
Workflow	GitHub feature branches + PR reviews, Postman, WhatsApp daily standups

Why this architecture fits

This project extends our existing Phase 1–2 codebase rather than starting over, so the architecture is proven: the React SPA already consumes our Flask REST API, and SQLAlchemy with Flask-Migrate means every schema change (the new users table, the user_id on favorites) ships as a reviewable migration instead of a hand-edited database. Enforcing access control in the backend decorators — not in the UI — means hiding a button is never mistaken for security. A five-person team maps naturally onto this stack's seams: models and decorators, API routes, the frontend auth layer, the favorites UI, and documentation can be built in parallel branches and integrated through pull requests.

Anticipated challenges

- **Ownership bugs (insecure direct object references).** The riskiest failure is a user reaching another user's favorite by guessing an ID. Mitigation: filter every query by the authenticated user's id and verify with a dedicated two-account test before demo.
- **Token lifecycle edge cases.** An expired or tampered token must fail cleanly, not crash the UI. Mitigation: the Axios 401 interceptor wipes the dead token and AuthContext falls back to logged-out, redirecting to /login.
- **CORS with the Authorization header.** Custom headers trigger preflight requests between the Vite dev server and Flask. Mitigation: Flask-CORS configured early and re-tested after deployment (a known cost from Phase 2).
- **localStorage token storage trade-off.** Convenient but exposed to XSS. Accepted for this project's scope, with a short expiry (8 hours) as mitigation; flagged as future hardening (httpOnly cookies).

- **Coordinating five parallel branches.** Auth is a dependency for everything, so merge order matters. Mitigation: build-order plan (models → backend auth → frontend auth → features), PR review before every merge.

Part 3 — Timeline and Scope

Phase 3 is timeboxed to **Thursday 9 July – Wednesday 15 July 2026** (six working days), extending the codebase delivered in Phases 1–2. Work is split across the team per the roles above; frontend and backend tracks run in parallel after the data model is agreed.

Dates	Est. time	Focus	Checkpoints
Thu 9 Jul	0.5 day	Business problem identification & project planning — kickoff, confirm roles, agree build order.	Research: sessions vs JWT-style tokens; itsdangerous vs flask-jwt-extended vs Flask-Login.
Thu 9 Jul	0.5 day	Database & model design — unified User with role, user-owned Favorite, unique constraint; first migration.	Team review of the ERD before any dependent code is written.
Fri 10 Jul	0.5 day	UI planning & wireframing — Login/Register layouts, navbar auth chip, protected My Favorites view.	—
Fri 10 – Sun 12 Jul	2 days	Backend implementation & auth — /auth endpoints, password hashing, token issuing, decorators, ownership-checked favorites CRUD.	Research: token expiry handling; Postman collection updated as endpoints land.
Sat 11 – Mon 13 Jul	2 days (parallel)	Frontend structure & fetches — Axios interceptors, AuthContext, ProtectedRoute, Login/Register pages, favorites UI.	Frontend built against mocks first; integrated once live endpoints are merged.
Mon 13 Jul	0.5 day	Project critique — PRs opened; every branch peer-reviewed; demo walkthrough of the auth flow to the whole team.	Feedback checkpoint: review comments logged as fixes.
Tue 14 Jul	1 day	Iterate on feedback, then debug & test — merge branches, resolve conflicts, full Postman pass, two-account ownership test.	Iteration checkpoint: rework from critique lands before final testing.
Wed 15 Jul	0.5 day	UI polish & error handling — loading/error states, responsive fixes, design-system alignment on auth pages.	—
Wed 15 Jul	0.5 day	Reflection & video — individual written reflections, README/ERD updates, recorded demo (register → login →	Final security checklist: hashed passwords, secrets in env vars, ownership verified.

Dates	Est. time	Focus	Checkpoints
		save favorite → My Favorites → admin CRUD).	

Topics researched or reviewed for this phase

- Session-based vs token-based authentication in decoupled SPA + API architectures (deciding factor: no shared origin, no server-side session store).
- Signed-token implementations: itsdangerous URLSafeTimedSerializer vs flask-jwt-extended — expiry, verification, and error handling.
- Secure password storage with werkzeug.security and why plaintext or reversible encryption is never acceptable.
- Axios interceptors for attaching credentials globally and centralising 401 handling.
- Ownership enforcement patterns (filtering by the authenticated user's id) to prevent insecure direct object references.

Scope statement.

In scope for Phase 3: registration, login, logout, session persistence across reloads, user-owned favorites with full CRUD, role-based admin access, and backend-enforced access control. Out of scope (future work): password reset emails, httpOnly-cookie token storage, refresh tokens, and expanding the curated dataset beyond the current countries and indicators.